JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Introduction

Outline

- Overview
- Course Topics
- Generative versus Discriminative Modeling
- Fundamental Generative Models
- Generative Adversarial Networks (GANs)
- Applications of Generative AI and Examples
- A Few Generative Models
- Generative Model Taxonomy
- Ethical Considerations

Overview

- What is Generative AI?
  - AI systems that generate new content (text, images, audio) by learning patterns from existing data
  - Examples: GANs, VAEs, Transformers
- Key Applications
  - Text Generation: Automated writing and chatbots
  - Image/Video Synthesis: Creating realistic visuals
  - Music/Audio Creation: AI-generated sounds
- Why is it Important?
  - Creativity & Automation: Content creation at scale
  - Efficiency: Faster design and data augmentation
- Challenges:
  - Ethical Concerns: Deepfakes, bias
  - Technical Hurdles: Model complexity, quality control
- **Note:** This entire slide (pg.3) was generated by ChatGPT-4 and remains unedited

Present Day

1956

Artificial Intelligence

- The field of computer science that seeks to create intelligent machines that can replicate or exceed human intelligence

1997

Machine Learning

- Subset of AI that enables machines to learn from existing data and improve upon that data to make decisions or predictions

2017

Deep Learning

- A machine learning technique in which layers of neural networks are used to process data and make decisions

2021

Generative AI

- Create new written, visual, and auditory content given prompts or existing data

Artificial Intelligence ⊃ Machine Learning ⊃ Deep Learning ⊃ Generative AI

Course Topics

- Review of ML
  - Core probability, statistics, linear algebra, GMM, HMM, and Autoregressive algs
- Autoencoders and more Review of ML
  - Variational Autoencoders and short review of supervised, unsupervised, and reinforcement learning concepts, review of Neural Networks
- Generative Adversarial Networks (GAN)
  - Adversarial learning, sample generation, application on FashionMNIST dataset
- Classical Generative AI and Autoregressive Models
  - Genetic algorithms, Gaussian Mixture Models (GMMs), Hidden Markov Models (HMMs), and Bayesian Networks

Course Topics

- Image and Video Generation
  - Image synthesis, latent space, Spatio-temporal GANs for video, optimize and evaluate using inception score (IS) and other metrics
- The Mathematical Model of a Transformer
  - Attention mechanism, Seq2Seq neural network architecture, Transformer model
- Transformers and Text Generation
  - Transformer, LLMs, LangChain, evaluation and optimization
- Advanced GAN Methods and Applications
  - Deep Convolutional GAN, Wasserstein GAN, Conditional GAN, ProGAN, StyleGAN, and other important GANs

Course Topics

- Audio and Music Generation
  - Audio synthesis using WaveNet, RNN, LSTM, encoding and decoding musical information, Mel-frequency Cepstral Coefficient (MFCC), perceptual evaluation
- Prompt Engineering
  - Explicit instructions, context, constraints, prompt design, LangChain
- Retrieval-Augmented Generation (RAG)
  - Overview of RAG, Retriever and Generator, evaluation metrics, LangChain
- Generative Agents
  - Multiple agents, self-reflection, Chatbots, virtual assistants, creative and simulating agents
- Ethics in AI and Generative Methods

Supervised vs Unsupervised Learning

Supervised Learning

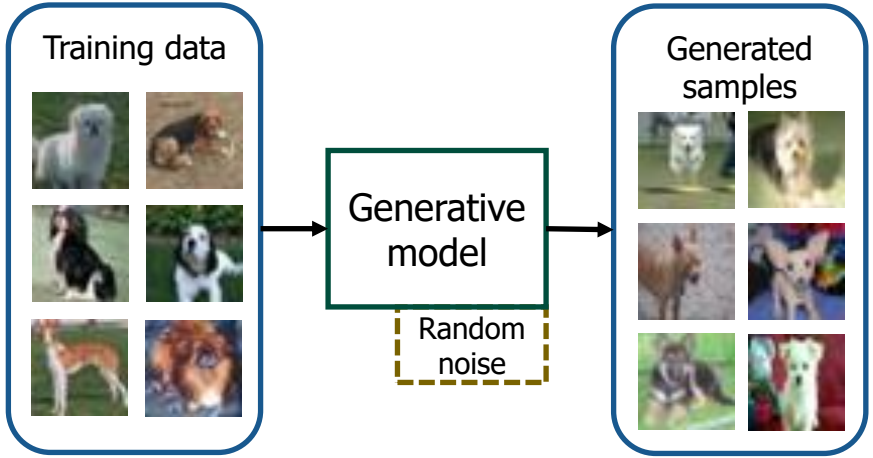
- Data: (X, y)
  - X is data, y is label
- Goal
  - Learn the mapping $\mathrm{f}: X \rightarrow y$
- Examples
  - Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

- Data: (X)
  - No labels
- Goal
  - Learn the underlying hidden **structure** of the data
  - Learn its density $\mathrm{p_X}(x)$
- Examples
  - Clustering, dimensionality reduction, feature learning, density estimation, etc.

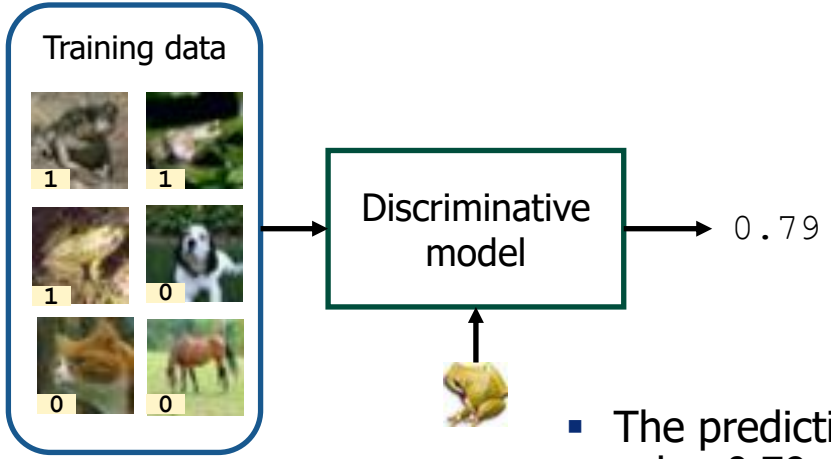
Generative Model

- Generative modeling is a machine learning approach focused on training models to generate new data that closely resembles a provided dataset
- Via training, the generative model is built
- Via sampling through random noise, samples are generated
  - Random noise is used to sample the latent space learned earlier
  - Creativity requires a creative training strategy!



Generative versus Discriminative Model

- Discriminative modeling uses labeled data to classify observations, like distinguishing frog images from others
  - A cohort or an experiment or the data collection created integer $y \in \{0,1\}$ labels
- The classifier can compute the probability $\mathrm{P}(y|X)$ to find the estimate $\hat{y}$
- Generative modeling, on the other hand, creates new images without needing labels



- The prediction value 0.79 infers the input image is likely a frog

Examples of Classical Models

Fundamental Generative Models

- **Expert Systems**
  - Apply domain-specific knowledge encoded as rules or facts in a knowledge base
  - Infer solutions to complex problems through logical reasoning
  - Similar to the decision-making process of human experts (e.g. in medicine)

- Expert System Algorithm

1. **Initialize** knowledge base K with domain-specific facts and rules
2. **Input** the problem P to be solved
3. **Initialize** solution S as empty
4. **Retrieve** relevant rules R from K
5. **For each** rule $r \in R$:

   If r's conditions are satisfied by P:

   Apply r to infer partial solution S_r

   Update S with S_r

   Else, skip to the next rule
6. **If** S is complete:

   Return S as the solution
7. **Else**, consult a human expert or additional knowledge to resolve P

Fundamental Generative Models

- **Genetic Algorithms**
  - Optimization techniques inspired by the principles of natural selection, utilizing processes like mutation, crossover, and selection to evolve solutions
  - Iteratively refine populations of candidate solutions based on a fitness function

- Genetic Algorithm for Optimization

1. **Initialize** population P with random candidate solutions
2. **Evaluate** the fitness of each candidate in P
3. **While** termination criteria not met do

   Select parent solutions from P based on fitness

   Apply crossover to select parents for offspring

   Apply mutation to offspring with probability

   Evaluate the fitness of each offspring

   Select the next generation population from the combined set of parents and offspring
4. **End while**
5. **Return** the best solution found

Fundamental Generative Models

- **Constraint Programming**
  - Define a set of constraints and variables
  - Search and infer to find solutions that satisfy all the constraints
  - Combinatorial optimization problems like scheduling and resource allocation

- Constraint Programming

1. **Define** a set of variables V
2. **Define** a set of constraints C over the vars V
3. **Initialize** an empty solution set S
4. **Apply** const propagation to shrink domain V
5. **While** there are unassigned variables in V do
   Select a variable v from V
   Select a value val from the domain of v
   If assigning val to v satisfies all relevant in C:
   Assign val to v and Add this partial to set S
   Else, backtrack to try a different assignment
6. **End while**
7. **If** all constraints in C are satisfied:
   Return the complete solution set S
8. **Else**, report no solution found

Fundamental Generative Models

- **Markov Chains**
  - Stochastic models that describe sequences of events
  - The probability of each event depends solely on the state of the previous event
  - Governed by transition probabilities in a memoryless process

- Markov Chain Model

1. **Define** the set of states S
2. **Define** the transition probability matrix $\mathbf{P}$, where $\mathbf{P}[i][j]$ represents the probability of transitioning from state i to state j
3. **Initialize** the current state $s \in S$
4. **Define** the no of steps N for the simulation
5. **For each** step $t = 1$ to N do:

   Sample the next state s_{next} from the transition probabilities of the current state s

   Set $s = s_{next}$

   Record the current state s
6. **End for**
7. **Return** the sequence of states as the result of the Markov chain simulation

Fundamental Generative Models

- **Normalizing Flows**
  - Transform a simple probability distribution into a more complex one by applying a sequence of invertible, differentiable mappings
  - Allowing efficient sampling and exact likelihood estimation

- Normalizing Flows for Density Estimation

1. **Initialize** base distribution p_Z (e.g., standard normal)
2. **Define** a sequence of n invertible, differentiable transformations $f_1, f_2, \dots, f_n$
3. **Sample** from base distribution $z \sim p_Z$
4. **For** $i = 1$ to n do:

   Apply transformation $z \leftarrow f_i(z)$
5. **End for**
6. **Compute** final transformed distribution p_X using the change of variables formula
7. **Return** transformed sample $x = z$ and its likelihood under p_X

Fundamental Generative Models

- **Diffusion Models**
  - Reverse a gradual noise-adding process, learning to denoise data step-by-step
  - Generate complex data distributions by sequentially refining random noise into structured outputs

- Sampling

1. **Initialize** $x_T \sim N(0, I)$ (random noise)
2. **For each** step $t = T, \ldots, 1$ do:

   Gradually denoise: $x_{t-1} = p_\theta(x_t, t)$
3. **End for**
4. **Return** generated sample x_0

- Diffusion Model

1. **Initialize** data distribution x_0 (e.g., training)
2. **Define** noise $q(\epsilon_t)$ to add noise gradually
3. **Define** denoising model p_θ (parameterized model to reverse the noise process)
4. **Set** total number of diffusion steps T
5. **For each** training iteration do:

   Sample data x_0 from the data distribution

   Sample a random time step $t \in \{1, \ldots, T\}$

   Add noise to the data: $x_t = q(x_0, t)$, where noise is added according to $q(\epsilon_t)$

   Predict the noise using the model: $\hat{\epsilon}_t = p_\theta(x_t, t)$

   Calculate loss btw predicted $\hat{\epsilon}_t$ and actual ϵ_t

   Update model params θ to minimize the loss
6. **End for**

Patterns, Data Abstractions

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Latent Space

- A low-dimensional, abstract representation of data where the underlying structure, patterns, or features are captured in a compressed form
  - A method of generalization
  - Usually, a transformation from the high-dimensional input data
    - Retains the most important information while discarding irrelevant details
- **Encoder** $f(\mathbf{x})$ transforms the data points $\mathbf{x} \in \mathbb{R}^D$ to $\mathbf{z} \in \mathbb{R}^d$ where $D \gg d$, i.e. $\mathrm{f}: x \to z$
- **Decoder** $g(\mathbf{z})$ inverse transforms $f(\mathbf{x})$ back to $\mathbf{x}$, i.e. $\mathrm{g}: z \to x$
- An optimization method minimizes the reconstruction loss $\mathrm{L}(\mathbf{x}, g(\mathbf{z}))$, e.g. L_1, L_2, etc.
- GAN methods and sequence-to-sequence utilize latent space in their encoding and decoding scheme
  - Mapping patterns are learned through this feature

Representation Learning

- Automatically discover useful features (representations) from high-dimensional data
  - to make tasks like classification, prediction, or generation easier (for us)
- Transform raw data (such as pixel values in an image or words in a sentence) into a set of meaningful and compact representations
- Example, describe the appearance to someone in a concise and meaningful way
  - Instead of conveying every pixel detail, describe high-level features like "blond hair" or "wearing glasses"
  - These representations are assumed to be known by the observer/recipient too
- A model can capture the underlying structure of the data
  - More effective at generalizing and making predictions
  - Even in scenarios it did not encounter before

Autoencoder

- A neural network learning low-dimensional representations (encodings) of data
- Encoder maps the input data to the latent space
- Decoder reconstructs the data from the latent space
- Helps dimensionality reduction, feature learning, and denoising

```
X_DIM= 28 # size of the images
class Autoencoder(nn.Module):
  def __init__(self, z_dim):
    super().__init__(); self.z_dim= z_dim
    self.encoder = Sequential(
      Linear(X_DIM*X_DIM, 128), ReLU(),
      Linear(128, 64), ReLU(),
      Linear(64, self.z_dim) )
    self.decoder = Sequential(
      Linear(self.z_dim, 64), ReLU(),
      Linear(64, 128), ReLU(),
      Linear(128, X_DIM*X_DIM) )
  def forward(self, _X):
    encoded = self.encoder(_X) # zval
    decoded = self.decoder(encoded)
    return decoded
```

Manifold Learning

- An unsupervised learning assumes high-dimensional data lies on/near a lower-dimensional manifold embedded in a higher-dimensional (topological) space
  - Discover this lower-dimensional structure (the manifold) to reduce the complexity
  - Dimensionality reduction
- A **manifold** locally resembles Euclidean space (e.g., a plane)
  - Possibly in a more complex global structure
  - Locally Euclidean
  - A line is a 1-dimensional manifold
  - A plane or the surface of a sphere is a 2-dimensional manifold
    - The sphere is 3D and has a higher level of complexity
  - The space we live in is a 3-dimensional manifold

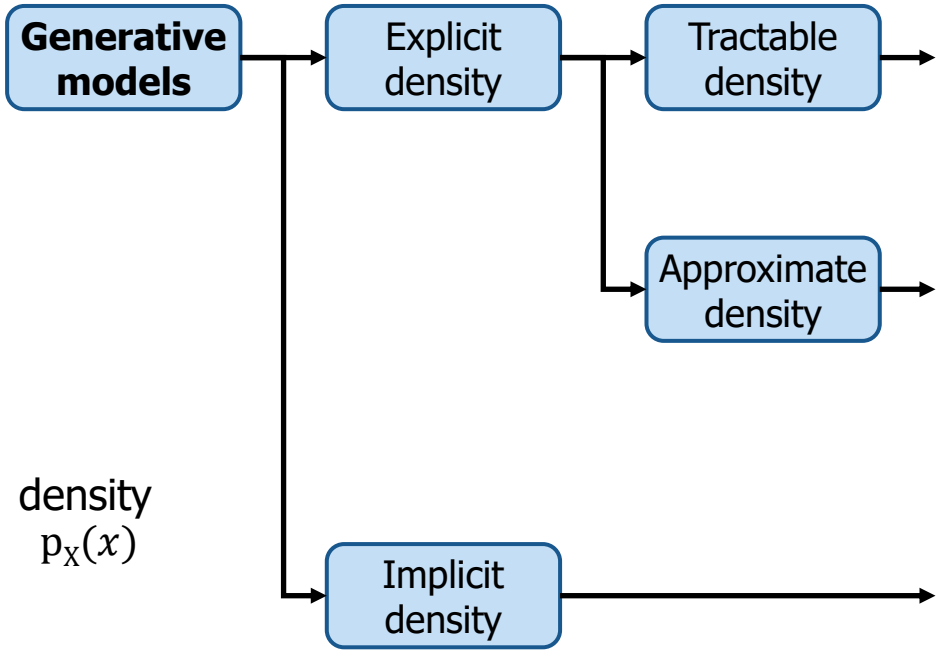
Manifold Learning Algorithms

- Manifold learning seeks a mapping $\phi: \mathbb{R}^D \to \mathbb{R}^d$ projects data points from the high-dimensional space to a low-dimensional space (again $D \gg d$)
  - Preserve the manifold's structure
  - Preservation distances, angles, or neighborhood relationships
  - Basically, the integrity of the original data in its lower-dimensional representation
- **Principal Component Analysis (PCA)**
  - Finds the directions $v_1, v_2, \ldots, v_d$ (eigenvectors) of the covariance matrix of the data that correspond to the largest eigenvalues (the principal components)
  - $z_i = V^{\mathrm{T}} x_i$

Manifold Algorithms

- **t-SNE** (t-Distributed Stochastic Neighbor Embedding)
  - In high dimensionality P, in low dimensionality Student's-t Q
  - Objective is to minimize the Kullback-Leibler (KL) divergence between P and Q
  - $\mathrm{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$
- **Isomap**
  - Construct a neighborhood graph using k-neighbors and then scale
  - Shortest paths are computed by Dijkstra, etc.
- **Locally Linear Embedding** (LLE)
  - For each data point x_i, find the weights w_{ij} and reconstruct x_i as a linear combination of its neighbors

Generative Model Taxonomy



- Autoregressive models
- Normalizing flow models, NICE, RealNVP
- PixelRNN/CNN, Glow

- Variational autoencoders
- Energy-based models
- Diffusion models
- Markov chain, Boltzmann machine

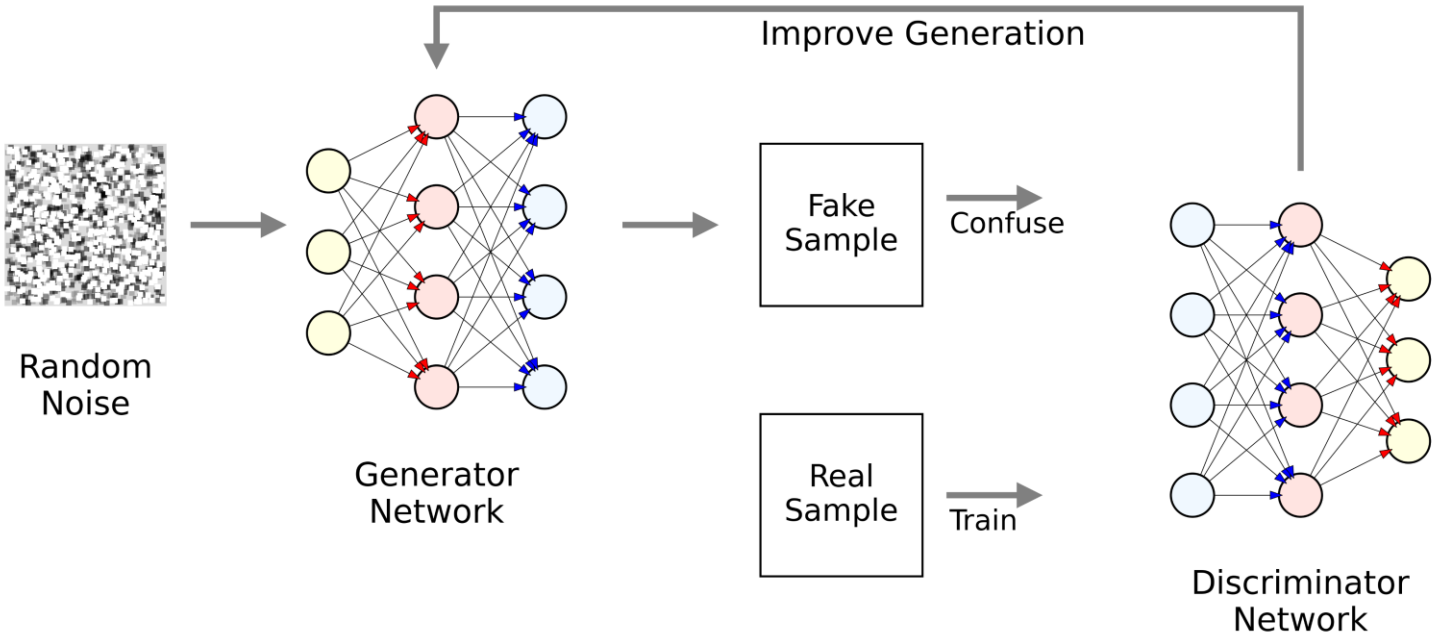
- Generative adversarial networks (GANs)
- Markov chain, GSN

- density $p_X(x)$

Generative Adversarial Networks (GANs)

- GANs have been applied to computer vision, natural language processing, time series synthesis, semantic segmentation; and can achieve plausible image generation, image-to-image translation, image super-resolution, image completion, etc.
- Traditional methods
  - A classification model maps high-dimensional data to low dimensions
  - A generative model maps low-dimensional data to high dimensions
- A novel approach combines two independent models, taking the output of a generative model as the input of the classification model
  - The classification model receives generated samples along with real samples
- Two optimizations going on in parallel
  - Similar to a two player game

Generative Adversarial Networks (GANs)



Advantages of GANs

- Adversarial training, direct feedback
  - The generator can continuously improve its outputs via discriminator feedback
  - Improved realism due to the ability of the generator to "fool" the discriminator
- Implicit density estimation
  - Allows GANs to capture complex, high-dimensional distributions
- Flexibility and expressiveness
  - GANS have better expressiveness due to implicit density
  - Generator/discriminator can tailor to specific tasks (e.g., StyleGAN, CycleGAN)
  - Discriminator can also perform categorization (semi-supervised learning)
- Sample quality, scalability, transfer learning, diverse sampling
  - Natural characteristics due to deep learning models

Applications of Generative AI in Business

- Credible writing
  - Respond to criticism to make the writing more fit for a purpose
- Correct code generated by AI models
- Technical materials, such as higher-resolution versions of medical images
- Out of the box or fine-tune them to perform a specific task
  - Learn a specific style in your business writing
- According to one study, companies spare 10% of revenue to marketing budget
  - Of that 80% is digital marketing, of which 45% is content creation
- AI adoption doubled in 5 years
- Over one million people signed up to use ChatGPT in just five days after launch

Ethical Issues and Challenges

Designated use	Misuse	Accidents from use	Structural effects on society	Misaligned, power seeking AIs
▪ New, smaller competitors, enabled by GenAI, entering and disrupting industries	▪ Mass production of misinformation and deepfakes	▪ AI hallucinations leading to incorrect decision making	▪ Mass unemployment of creators of content and code	▪ Power-seeking AI model with goals not aligned with humanity

Intended → **Unintended**

Build Trustworthy Generative AI

Transparency

Make sure the output is always recognizable as machine-generated

Privacy and Security

Be careful with intellectual property and private data

Bias and Accuracy

Make use of reinforcement learning through human feedback

Governance

Keep in beta, educate and embrace regulation

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- ChatGPT-4o, 14 May version, OpenAI, 2024, chat.openai.com/chat.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.